

Additional Exercises to Exercise Set 1

Topic 1: Basic Class Creation (Based on Exercise 1)

Exercise 1.1

Easy

Create a class `Book` with private data members `title` (string), `author` (string), and `pages` (int). Include public member functions to set and get each of these values separately.

Expected Implementation Details:

Define three private members as specified. Create six public methods: `setTitle(string t)`, `getTitle()`, `setAuthor(string a)`, `getAuthor()`, `setPages(int p)`, `getPages()`. In `setPages()`, validate that pages is positive. Add a `displayInfo()` method that prints book details in format "Title by Author, Pages pages". Initialize all members in a default constructor. Test encapsulation by verifying direct member access fails.

Exercise 1.2

Easy

Design a class `Temperature` with a private data member `celsius`. Create public member functions `setCelsius()`, `getCelsius()`, `getFahrenheit()`, and `getKelvin()` that perform appropriate conversions.

Expected Implementation Details:

Store temperature as private double `celsius`. Implement `setCelsius(double c)` with validation for absolute zero (-273.15°C). `getFahrenheit()` returns $(\text{celsius} * 9/5) + 32$. `getKelvin()` returns $\text{celsius} + 273.15$. Consider adding `setFahrenheit(double f)` and `setKelvin(double k)` that internally convert to Celsius. Include a constructor that accepts initial temperature in Celsius. Test all conversion formulas with known values.

Exercise 1.3

Easy

Implement a class `Time` with private data members `hours`, `minutes`, and `seconds`. Include public setter and getter functions for each member with basic validation (e.g., minutes and seconds should be 0-59).

Expected Implementation Details:

Use private int members for all three fields. Validate: hours (0-23), minutes (0-59), seconds (0-59). If invalid values provided, either clamp to valid range or reject (document choice). Add `setTime(int h, int m, int s)` to set all at once. Include `addSeconds(int s)` that properly handles overflow (e.g., adding 90 seconds increases minutes). Provide `display()` showing time as "HH:MM:SS" with leading zeros. Constructor should default to 00:00:00.

Topic 2: Default Constructor (Based on Exercise 2)

Exercise 2.1

Easy

Write a class `Product` with private members `name` (string), `price` (double), and `quantity` (int). Create a default constructor that initializes name to "Unnamed", price to 0.0, and quantity to 0.

Expected Implementation Details:

Define default constructor `Product()` using initialization list: `Product() : name("Unnamed"), price(0.0), quantity(0) {}`. Add getter methods for all members. Include `getTotalValue()` returning `price × quantity`. Add a `display()` method showing product details. Test by creating objects without arguments and verify default values. Consider whether these defaults make business sense for your application.

Exercise 2.2

Easy

Create a class `Point2D` with private members `x` and `y` (both double). Implement a default constructor that initializes both coordinates to 0.0. Add a display function to show the point as (x, y).

Expected Implementation Details:

Use initialization list in default constructor. Add methods: `setX()`, `setY()`, `getX()`, `getY()`. Include `distanceFromOrigin()` returning `sqrt(x2 + y2)`. Add `distanceTo(const Point2D& other)` calculating distance between points. Display format should be "(3.14, 2.71)" with appropriate precision. Consider adding a method to move the point by dx, dy.

Exercise 2.3

Easy

Design a class `Vehicle` with private members `brand` (string), `model` (string), and `year` (int). Create a default constructor that initializes `brand` to "Unknown", `model` to "Unknown", and `year` to the current year (2024).

Expected Implementation Details:

Initialize `year` to 2024 (or use a constant `CURRENT_YEAR`). Add public methods to access and modify all members with appropriate validation (year should be reasonable, e.g., 1900-2024). Include `getAge()` returning current year minus vehicle year. Add `displayInfo()` showing "YYYY Brand Model" format. Test edge cases like very old or future years. Consider adding vehicle type (car, truck, etc.) with appropriate default.

Topic 3: Parameterized Constructor (Based on Exercise 3)

Exercise 3.1 Easy

Modify the `Product` class to include a parameterized constructor that accepts name, price, and quantity as parameters. Ensure the constructor validates that price and quantity are non-negative.

Expected Implementation Details:

Add constructor `Product(string n, double p, int q)`. In the constructor body or initialization list, check if `price < 0` or `quantity < 0`. If invalid, set to 0 and optionally print warning. Keep the default constructor so class now has two constructors. Test with: valid data, negative price, negative quantity, empty string name. Consider using `assert()` or throwing exception for invalid values (mention but implementation optional).

Exercise 3.2 Easy

Update the `Point2D` class to include a parameterized constructor accepting x and y coordinates. Add another constructor that accepts a single value and uses it for both x and y.

Expected Implementation Details:

Implement two additional constructors: `Point2D(double x, double y)` and `Point2D(double value)`. The single-parameter constructor creates points on the line $y=x$. Now class has three constructors total. Test constructor overloading with: `Point2D p1; (0,0)`, `Point2D p2(3.5, 2.1);`, `Point2D p3(5.0); (5,5)`. Document why single-value constructor might be useful (e.g., square grid points).

Exercise 3.3

Easy

Enhance the `Vehicle` class with a parameterized constructor that accepts brand, model, and year. Validate that the year is between 1900 and the current year.

Expected Implementation Details:

Add `Vehicle(string b, string m, int y)`. Validate year: if $y < 1900$, set to 1900; if $y > 2024$, set to 2024. Consider printing warning for out-of-range values. Use initialization list for efficiency. Keep default constructor for backward compatibility. Test boundary values: 1899, 1900, 2024, 2025. Add static method `isValidYear(int y)` for reusable validation logic.

Topic 4: Access Specifier Practice (Based on Exercise 4)

Exercise 4.1

Easy

Create a class `EmailAccount` with private members `email` and `password`. Include public functions `setEmail()`, `getEmail()`, and `changePassword()` (which requires the old password). The password should never be retrievable.

Expected Implementation Details:

No `getPassword()` method should exist. `changePassword(string oldPass, string newPass)` returns `bool` - `true` if `oldPass` matches stored password. Consider storing password hash instead of plain text (mention concept). Add `validateLogin(string attemptedPassword)` returning `bool`. Include email format validation in `setEmail()` (must contain `@` and `.`). Initialize password to empty string in constructor. Test that password remains completely private.

Exercise 4.2

Easy

Design a class `GradeBook` with private array `grades[10]` and private member `count`. Create public functions `addGrade()`, `getAverage()`, and `getHighestGrade()`. Ensure direct access to the `grades` array is not possible.

Expected Implementation Details:

Initialize `count = 0` in constructor, and optionally initialize all grades to 0. `addGrade(double grade)` should check if `count < 10` before adding, validate grade range (0-100). `getAverage()` returns `sum/count` (handle `count=0` case). `getHighestGrade()` iterates through valid grades only. No method should return pointer/reference to the `grades` array. Add `getGrade(int index)` with bounds checking. Test array privacy by trying `gradeBook.grades[0]` (should fail).

Exercise 4.3

Intermediate

Implement a class `Counter` with a private static member `totalCount` and private member `instanceCount`. Provide public functions `increment()`, `getCount()`, and a static function `getTotalCount()`.

Expected Implementation Details:

Declare `static int totalCount` in class, define outside: `int Counter::totalCount = 0;`. Each `increment()` increases both `instanceCount` and `totalCount`. Constructor sets `instanceCount = 0`. `getTotalCount()` is static, accessed via `Counter::getTotalCount()`. Add `reset()` to set `instanceCount = 0` (`totalCount` unchanged). Destructor might decrement `totalCount` by `instanceCount`. Test with multiple `Counter` objects to verify shared `totalCount`.

Topic 5: Constructor Overloading (Based on Exercise 5)

Exercise 5.1

Easy

Create a class `Box` with private members `length`, `width`, and `height`. Implement four constructors:

- Default constructor (all dimensions = 1)
- Single parameter constructor (creates a cube)
- Three parameter constructor
- Copy constructor

Expected Implementation Details:

Default: `Box() : length(1), width(1), height(1) {}`. Single parameter: `Box(double side) : length(side), width(side), height(side) {}` creates cube. Three parameter: `Box(double l, double w, double h)` with validation for positive values. Copy: `Box(const Box& other)` copies all dimensions. Add `getVolume()` and `getSurfaceArea()` methods. Test all constructors and verify each creates correct box type.

Exercise 5.2

Intermediate

Design a class `String` with a private character pointer. Implement:

- Default constructor (empty string)
- Parameterized constructor accepting C-string
- Parameterized constructor accepting a single character
- Copy constructor (deep copy)

Expected Implementation Details:

Private member: `char* str`. Default: allocate for empty string (''). C-string constructor: allocate `strlen(s)+1` bytes, `strcpy` content. Single char: allocate 2 bytes for char and '\0'. Copy constructor MUST perform deep copy: allocate new memory and copy content, not just pointer. Include `length()` method and `display()`. Destructor should `delete[] str`. Test deep copy by modifying copy without affecting original.

Exercise 5.3

Easy

Create a class `Matrix2x2` with a private 2D array. Implement constructors:

- Default constructor (identity matrix)
- Single parameter constructor (scalar matrix)
- Four parameter constructor (all elements)
- Copy constructor

Expected Implementation Details:

Store as `double matrix[2][2]`. Default creates identity: `[[1,0],[0,1]]`. Scalar constructor with parameter `k` creates `[[k,0],[0,k]]`. Four parameter takes elements in row-major order: `(a,b,c,d)` creates `[[a,b],[c,d]]`. Copy constructor copies all 4 elements. Add `determinant()` returning `ad-bc`. Include `multiply(const Matrix2x2& other)`. `Display` shows matrix in 2x2 format with aligned columns.

Topic 6: Initialization List (Based on Exercise 6)

Exercise 6.1

Intermediate

Create a class `Account` with const member `accountNumber`, reference member `branch`, and regular members `holderName` and `balance`. Use member initialization list to properly initialize all members.

Expected Implementation Details:

Constructor: `Account(int accNum, string& branchRef, string name, double bal) : accountNumber(accNum), branch(branchRef), holderName(name), balance(bal) {}`. The const and reference members **MUST** be in initialization list. Create a `Branch` string variable in `main()` to pass as reference. Attempting to change `accountNumber` after construction should give compiler error. Add methods to display account info and modify balance (but not account number).

Exercise 6.2

Intermediate

Design a class `Node` for a linked list with const member `nodeID`, reference member `data`, and pointer member `next`. Demonstrate proper initialization of all members using initialization list.

Expected Implementation Details:

Template or use specific type like `int` for data. **Constructor:** `Node(int id, int& dataRef, Node* n = nullptr) : nodeID(id), data(dataRef), next(n) {}`. The reference member creates an alias to external data. Include static counter for auto-generating unique `nodeIDs`. Test by creating nodes where data references variables that might change. Show that changing the referenced variable affects what the node sees.

Exercise 6.3

Intermediate

Implement a class `Configuration` with multiple const members: `maxUsers`, `timeout`, and `version`, plus a reference member `configFile`. Show how to initialize these using initialization list with different constructors.

Expected Implementation Details:

Provide multiple constructors: one taking all parameters, one with defaults for some const members. Example: `Configuration(int max, int time, float ver, string& file) : maxUsers(max), timeout(time), version(ver), configFile(file) {}`. Add constructor that reads from the referenced config file to set other values. Show that initialization list is the only way to set const members. Include methods to display configuration but not modify const values.

Topic 7: Multiple Constructors with Validation (Based on Exercise 7)

Exercise 7.1

Intermediate

Design a class `PhoneNumber` with private members for country code, area code, and number. Implement:

- Default constructor (sets to a valid default number)
- Parameterized constructor with validation (valid country codes, area codes)
- Copy constructor

Include format validation and a display function.

Expected Implementation Details:

Store as three integers or strings. Default could be +1-555-1234567. Validate: country code 1-999, area code not starting with 0 or 1, number 7-10 digits. Constructor should handle formats: "+91-22-12345678" or separate parameters. Include `isValid()` private method for validation logic. Display format: "+CC-AAA-NNNNNNNN". Handle invalid input by setting to default values. Test with various international formats.

Exercise 7.2

Intermediate

Create a class `IPv4Address` with four octets. Implement:

- Default constructor (127.0.0.1)
- Parameterized constructor with validation (each octet 0-255)
- String constructor that parses "192.168.1.1" format
- Copy constructor

Expected Implementation Details:

Store as `unsigned char octets[4]` or four int members. String constructor uses string parsing (find '.', substring, stoi). Validate each octet ≤ 255 . Include `isPrivate()` checking for 10.x.x.x, 172.16-31.x.x, 192.168.x.x. Add `toString()` returning dotted format. Consider special addresses: 0.0.0.0 (invalid), 255.255.255.255 (broadcast). Test parsing malformed strings like "192.168.300.1" or "192.168.1".

Exercise 7.3

Intermediate

Design a class `CreditCard` with card number, expiry date, and CVV. Implement multiple constructors with validation for:

- Valid card number length (16 digits)
- Future expiry date
- 3-digit CVV

Include appropriate error handling.

Expected Implementation Details:

Store card number as string (to preserve leading zeros), expiry as month/year integers, CVV as string. Validate: card number exactly 16 digits (use regex or loop), expiry month 1-12 and year $\geq$ current year/month, CVV exactly 3 digits. Include Luhn algorithm for card number validation (optional advanced feature). Mask display: show only last 4 digits "****-****-****-1234". Constructor could accept "4532-1234-5678-9012" format with dashes.

Topic 8: Private Constructor Pattern (Based on Exercise 8)

Exercise 8.1

Intermediate

Create a class `Logger` that implements singleton pattern with a private constructor. Include methods `getInstance()`, `log(string message)`, and maintain a single log file throughout the application.

Expected Implementation Details:

Private constructor opens/creates log file. Static member: `Logger* instance = nullptr`. `getInstance()` creates instance on first call using `new`. Include timestamp in log entries. Consider thread safety issues (mention, don't implement). Add log levels: INFO, WARNING, ERROR. Private copy constructor and assignment operator to prevent copying. Destructor closes file. Test that multiple `getInstance()` calls return same pointer.

Exercise 8.2

Intermediate

Design a class `ConfigurationManager` with private constructor that loads settings from a file. Ensure only one instance exists and provide methods to read/update configuration values.

Expected Implementation Details:

Store settings in a map or struct. Private constructor reads config file (e.g., key=value format). Methods: `getString(key)`, `getInt(key)`, `setValue(key, value)`, `saveToFile()`. Handle missing keys gracefully with defaults. Include `reload()` to re-read file. Use lazy initialization: load file only when first accessed. Consider using a simple format like "database_host=localhost\nport=3306".

Exercise 8.3

Difficult

Implement a class `ResourcePool` with private constructor that manages a limited number of resources (e.g., database connections). Use static method to control access and limit the number of instances to a fixed pool size.

Expected Implementation Details:

Not a singleton but a multiton pattern. Static array/vector of instances with fixed `MAX_SIZE`. Static method `acquire()` returns available instance or nullptr if all busy. `release()` returns instance to pool. Each instance has `inUse` flag. Private constructor initializes resource. Include timeout mechanism for acquiring resources. Test with more requests than pool size. Consider implementing reference counting for automatic release.

Topic 9: Complex Constructor Chain (Based on Exercise 9)

Exercise 9.1

Difficult

Create a hierarchy: `Vehicle` (base) → `Car` (derived) → `ElectricCar` (derived).

- `Vehicle`: `engineNumber` (const), `manufacturer`, `year`
- `Car`: `doors`, `transmission` type, `fuel` type
- `ElectricCar`: `battery` capacity, `charging` time, `range`
- Implement all constructor types with proper chaining
- Use static member for vehicle count

Expected Implementation Details:

`Vehicle` constructor: `Vehicle(string engNum, string mfr, int yr) : engineNumber(engNum), manufacturer(mfr), year(yr) { vehicleCount++; }`.
`Car` must call `Vehicle` constructor: `Car(...) : Vehicle(engNum, mfr, yr), doors(d), transmission(t), fuelType(f) {}`. `ElectricCar` calls `Car` with `fuelType="Electric"`. Each class needs default, parameterized, and copy constructors. Copy constructors must chain properly and increment `vehicleCount`. Static `getVehicleCount()` in `Vehicle` class.

Exercise 9.2

Difficult

Design hierarchy: `Account` → `SavingsAccount` → `FixedDepositAccount` .

- `Account`: `accountID` (const), holder name, balance
- `SavingsAccount`: interest rate, minimum balance
- `FixedDepositAccount`: tenure, maturity amount, penalty rate
- Demonstrate constructor delegation and initialization lists

Expected Implementation Details:

`Account` uses static counter for unique IDs. `SavingsAccount` constructor validates minimum balance $\leq$ initial balance. `FixedDepositAccount` calculates maturity amount in constructor based on balance, rate, and tenure. Each derived constructor must explicitly call base constructor. Include protected methods in `Account` for balance modifications. Copy constructor in `FixedDepositAccount` must recalculate maturity amount. Test complete chain with all constructor types.

Exercise 9.3

Difficult

Implement: `Shape` → `Shape2D` → `Rectangle` → `Square`.

- Shape: `shapeID` (const), color
- Shape2D: x, y coordinates
- Rectangle: width, height
- Square: side length
- Show how Square constructor delegates to Rectangle
- Include static shape counter

Expected Implementation Details:

Square constructor calls Rectangle with: `Square(double s, ...) : Rectangle(s, s, ...)` making width=height=side. Each class in hierarchy has `area()` method (virtual in real implementation, but just override here). Shape constructor generates unique ID from static counter. Shape2D adds position. Include `move(dx, dy)` method. Copy constructors must maintain unique IDs (generate new ID, not copy). Test that Square correctly behaves as special Rectangle.

Topic 10: Resource Management with RAII (Based on Exercise 10)

Exercise 10.1

Difficult

Create a class `FileHandler` that:

- Opens a file in constructor (`FILE*` as private member)
- Implements copy constructor with proper handling
- Implements destructor to close file
- Provides `read()`, `write()`, and `seek()` methods
- Handles errors appropriately
- Bonus: Implement move semantics

Expected Implementation Details:

Constructor: `FileHandler(string filename, string mode)` opens with `fopen()`. Store `FILE*` pointer privately. Destructor: `if(file) fclose(file);`. Copy constructor has choices: deep copy (open same file again), shallow copy with reference counting, or disable copying. Implement `read(buffer, size)`, `write(data, size)`, `seek(offset, origin)`. Check file pointer before operations. Move constructor (bonus): transfer ownership by copying pointer and setting source to `nullptr`.

Exercise 10.2

Difficult

Design a class `NetworkConnection` that:

- Establishes socket connection in constructor
- Manages socket descriptor (private member)
- Implements deep copy constructor
- Closes connection in destructor
- Provides `send()` and `receive()` methods
- Implements timeout handling
- Bonus: Add SSL/TLS support management

Expected Implementation Details:

Constructor creates socket and connects to host:port. Store socket descriptor as int. Destructor calls `close()` on socket. Copy constructor creates new connection to same host:port (deep copy of connection, not socket descriptor). Include `isConnected()` method. `send()` and `receive()` handle partial transfers. Implement timeout with `setsockopt()`. Error handling: check all system calls, throw or return error codes. Consider non-blocking mode for bonus points.

Exercise 10.3

Difficult

Implement a class `ThreadPool` that:

- Creates worker threads in constructor
- Manages thread handles (private vector)
- Implements proper copy constructor (or delete it)
- Joins all threads in destructor
- Provides task submission methods
- Handles thread synchronization
- Bonus: Implement work-stealing queue

Expected Implementation Details:

Constructor: `ThreadPool(int numThreads)` creates threads in detached or joinable state. Store thread objects/IDs in vector. Destructor sets stop flag and joins all threads. Copy constructor should be deleted (threads can't be copied). Task queue with mutex protection. `submit(function)` adds to queue and notifies threads. Worker function loops waiting for tasks. Use condition variables for efficiency. Include `shutdown()` method for graceful termination. This tests understanding of RAI with system resources.